

23.自动生成部署C代码

1. 基本介绍

RKNN-Toolkit2 2.0.0版本新增 Codegen 接口，用于生成模型部署代码，简化开发者的上手难度。Codegen基于 CAPI 零拷贝接口进行二次封装，接口风格与 RKNN_Model_Zoo 中的 demo 一致。生成的部署代码可用于直接测试性能、验证精度，开发者也可以基于生成的代码进行二次开发。

举例如下：

代码块

```
1 ret = rknn.codegen(output_path='./rknn_app_demo',
2                     inputs=['./mobilenet_v2/dog_224x224.jpg'],
3                     overwrite=True)
```

- 调用 codegen 接口前，必须先调用 rknn.export 接口保存RKNN模型。
- output_path 为输出文件夹目录，用户可配置目录名称。
- inputs 填写模型输入的路径列表，允许不填。有效文件格式为 jpg / png / npy ，以 npy 文件为输入时， npy 数据的维度信息应与模型输入的维度信息保持一致。
- overwrite 设为 True 时，会覆盖 output_path 指定目录下的文件。默认值为 False 。
- 生成部署代码后，请参考生成目录下的 README.md 文档说明进行编译、测试。
- 若 inputs 填入有效值，部署代码示例在推理后，会评估C API接口与RKNN-Toolkit2模拟器之间的推理结果差异，评估方式为对比每一个输出的余弦相似度。
- 无NPU硬件平台限制，要求板端系统为Linux或Android。
- 支持量化、非量化模型。

2. 代码测试

14_codegen

代码块

```
1 import numpy as np
2 import cv2
3 from rknn.api import RKNN
4
5 def show_outputs(outputs):
```

```

6     output = outputs[0].reshape(-1)
7     index = sorted(range(len(output)), key=lambda k : output[k], reverse=True)
8     fp = open('./labels.txt', 'r')
9     labels = fp.readlines()
10    top5_str = 'mobilenet_v2\n-----TOP 5-----\n'
11    for i in range(5):
12        value = output[index[i]]
13        if value > 0:
14            topi = '[:>3d]] score:{:.6f} class:"{}"\n'.format(index[i],
value, labels[index[i]].strip().split(':')[1])
15        else:
16            topi = '[-1]: 0.0\n'
17        top5_str += topi
18    print(top5_str.strip())
19
20    def load_and_preprocess_image(image_path, target_size=(224, 224)):
21        """
22        加载并预处理图片
23        Args:
24            image_path: 图片路径
25            target_size: 目标尺寸 (width, height)
26        Returns:
27            numpy array: 预处理后的图片, shape=(1, H, W, 3)
28        """
29        # 读取图片
30        img = cv2.imread(image_path)
31        if img is None:
32            raise ValueError(f"Failed to load image: {image_path}")
33
34        # BGR转RGB (模型期望RGB输入)
35        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
36
37        # Resize到目标尺寸
38        img = cv2.resize(img, target_size)
39
40        # 转换为NHWC格式 (1, H, W, 3)
41        img = np.expand_dims(img, axis=0).astype(np.float32)
42
43        return img
44
45    if __name__ == '__main__':
46
47        # Create RKNN object
48        rknn = RKNN(verbose=False)
49
50        # Pre-process config
51        print('--> Config model')

```

```
52     rknn.config(mean_values=[103.94, 116.78, 123.68], std_values=[58.82,
53     58.82, 58.82], quant_img_RGB2BGR=True, target_platform='rk3588')
54
55     # Load model
56     print('--> Loading model')
57     ret = rknn.load_caffe(model='./mobilenet_v2_deploy.prototxt',
58     blobs='./mobilenet_v2.caffemodel')
59     if ret != 0:
60         print('Load model failed!')
61         exit(ret)
62     print('done')
63
64     # Build model
65     print('--> Building model')
66     ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
67     if ret != 0:
68         print('Build model failed!')
69         exit(ret)
70     print('done')
71
72     # Export rknn model
73     print('--> Export rknn model')
74     ret = rknn.export_rknn('./mobilenet_v2.rknn')
75     if ret != 0:
76         print('Export rknn model failed!')
77         exit(ret)
78     print('done')
79
80     # print('--> Generate cpp demo')
81     # ret = rknn.codegen(output_path='./rknn_app_demo', inputs=
82     ['./dog_224x224.jpg'], overwrite=True)
83     # if ret != 0:
84     #     print('Generate cpp demo failed!')
85     #     exit(ret)
86     # print('done')
87
88     # ===== 推理测试部分 =====
89     print('--> Init runtime environment')
90     ret = rknn.init_runtime()
91     if ret != 0:
92         print('Init runtime environment failed!')
93         exit(ret)
94     print('done')
95
96     # 加载并预处理图片
97     print('--> Loading and preprocessing image')
```


3. 体验生成的C代码

编译

代码块

```
1 export RKNPU2=/home/topeet/rknn/02_rknn-toolkit2/rknn-toolkit2-master/rknpu2
2
3 export GCC_COMPILER=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-
  linux-gnu/bin/aarch64-none-linux-gnu
4
5 ./build_linux.sh rk3588
```

推送到开发板端

代码块

```
1 scp -r rknn_app_demo_Linux/ root@192.168.137.142:/userdata
```

板端推理

代码块

```
1 export LD_LIBRARY_PATH=./lib/
2 ./rknn_app_demo ./mobilenet_v2.rknn ./data/inputs/dog_224x224.jpg
```

查看结果

```
root@topeet:/userdata/rknn_app_demo_Linux# ./rknn_app_demo ./mobilenet_v2.rknn ./data/inputs/dog_224x224.jpg
rknn_app_query_model_info: model input num: 1, output num: 1
INPUTS:
[NORMAL]: index=0, name=data, n_dims=4, dims=[1,224,224,3,], n_elems=150528, size=150528, w_stride=224, size_with_strid
e=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=-13, scale=0.018317
[NATIVE]: index=0, name=data, n_dims=4, dims=[1,224,224,3,], n_elems=150528, size=150528, w_stride=224, size_with_strid
e=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=-13, scale=0.018317
OUTPUTS:
[NORMAL]: index=0, name=prob, n_dims=4, dims=[1,1000,1,1,], n_elems=1000, size=2000, w_stride=0, size_with_stride=16000
, fmt=NCHW, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000
[NATIVE]: index=0, name=prob, n_dims=5, dims=[1,125,1,1,8,], n_elems=1000, size=2000, w_stride=0, size_with_stride=2000
, fmt=NC1HWC2, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000
E RKNN: [17:51:39.786] rknn_set_input_shapes error, rknn model is static shape type, please export rknn with dynamic_shape
s

LOAD INPUTS
input[0] - IMG: ./data/inputs/dog_224x224.jpg

RUNNING RKNN
loop[0] time: 3.073000 ms
Average time: 3.083000 ms

CHECK OUTPUT
check all output to './data/outputs/runtime'
with golden data in './data/outputs/golden'
output[0] - prob:
rknn_layout_convert: src->fmt=NC1HWC2(1,125,1,1,8,), dst->fmt=NCHW(1,1000,1,1,)
rknn_dtype_convert: convert from FP16 to FP32
cosine similarity: 0.998830
root@topeet:/userdata/rknn_app_demo_Linux#
```

这个警告可以忽略，不影响推理结果。它只是告诉你模型是固定输入形状的（静态shape），不是动态shape。你的模型输入是 `[1,224,224,3]`，输入图片也是224x224，所以完全没问题。